

Bloc 3 TP 2 - Test d'intrusion

Contexte	2
Les méthodologies	2
La méthodologie utilisée	3
Les machines nécessaires	3
Préparation de la machine victime (Windows 7)	5
Vérification du réseau entre les deux machines	7
Phase I : la reconnaissance	10
La documentation avec CherryTree	10
Les outils de reconnaissance de Kali	14
Reconnaissance avec SpiderFoot	15
Un obstacle : l'accès à Internet et le proxy	18
Phase II : le scanning	21
Le scanner de ports Nmap	22
Le scanner de vulnérabilités Nessus	26
Conclusion	31

Contexte

Ce TP est une introduction au **test d'intrusion**, aussi appelé le “**pentest**”.

On va d'abord voir la méthodologie que l'on suit, puis on va préparer les machines nécessaires : un attaquant sous **Kali Linux** et une victime sous **Windows 7**, que l'on placera dans le même réseau.

Les méthodologies

Dans le milieu de la sécurité, plusieurs méthodologies servent de référence pour un test d'intrusion. Les principales sont **OWASP**, **PTES**, **OSSTMM** et **NIST**.

Exercice 1 : Faire une recherche sur chacun de ces sigles.

- **OWASP** (Open Worldwide Application Security Project) : une fondation à but non lucratif créée en **2001**, spécialisée dans la sécurité des **applications web**. Elle met à disposition gratuitement le “**Top 10**” des risques web, des guides de test et des outils comme **ZAP**.
- **PTES** (Penetration Testing Execution Standard) : un standard qui décrit le déroulement d'un test d'intrusion. Il est découpé en **7 phases**, du pré-engagement jusqu'au rapport.
- **OSSTMM** (Open Source Security Testing Methodology Manual) : un manuel open source maintenu par l'**ISECOM**. Il propose une approche mesurable des audits, sur plusieurs canaux (humain, physique, sans-fil, télécoms et réseau).
- **NIST** (National Institute of Standards and Technology) : un organisme officiel américain qui publie des normes. Pour les tests de sécurité, on utilise surtout son guide **SP 800-115**.

La méthodologie utilisée

Pour nos tests d'intrusion, on utilise une méthodologie plus simple, en plusieurs étapes :

- **Négociation** : on définit avec le client le périmètre (le “**scope**”) et ce que l'on a le droit de faire.
- **Reconnaissance** : on récolte un maximum d'informations sur la cible avec des outils open source.
- **Scanning** : une reconnaissance plus poussée et plus agressive, avec des outils comme **nmap**.
- **Gain d'accès** : on utilise ces informations pour pénétrer le système de la cible.
- **Maintien d'accès** : on met en place ce qu'il faut pour rester présent sur le réseau.
- **Nettoyage des traces** : on supprime tout ce que l'on a laissé derrière nous.
- **Rapport** : on rédige le rapport détaillé, pour protéger l'infrastructure du client.

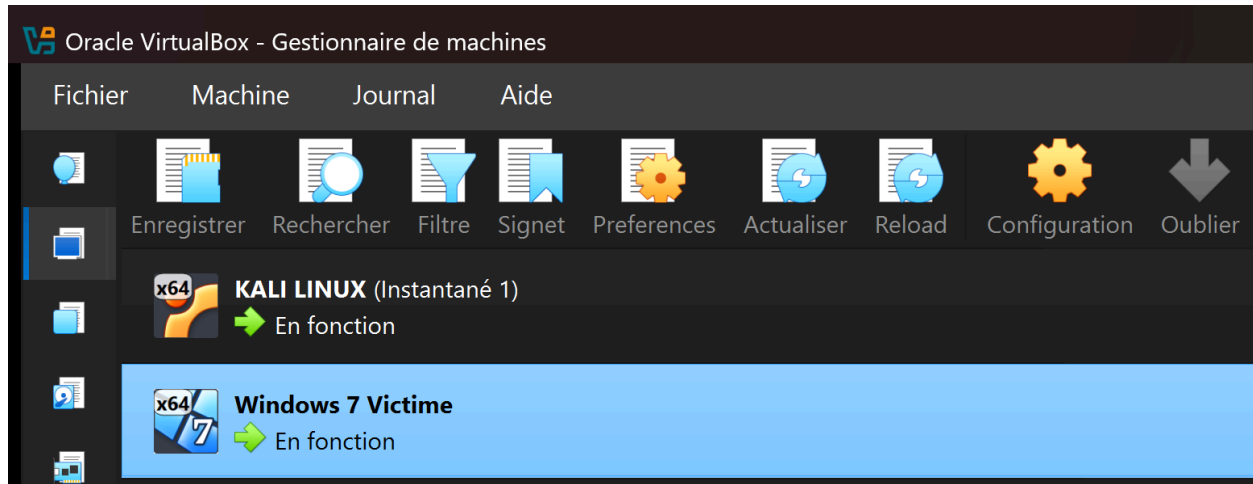
Les machines nécessaires

Pour ces tests, on a besoin de deux machines virtuelles :

- L'**attaquant** : **Kali Linux**, déjà installé lors du **TP1**.
- La **victime** : **Windows 7 SP1** Édition Intégrale.

Exercice 2 : Préparer ces machines et vérifier qu'elles sont dans le même réseau.

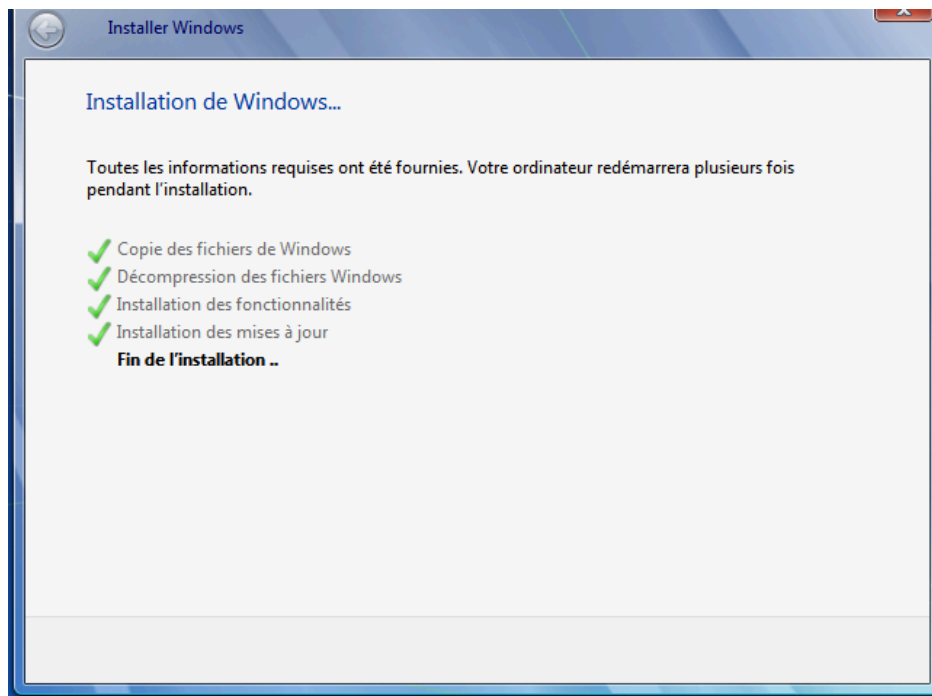
On retrouve bien nos deux machines dans le **Gestionnaire VirtualBox**, toutes les deux “En fonction” :



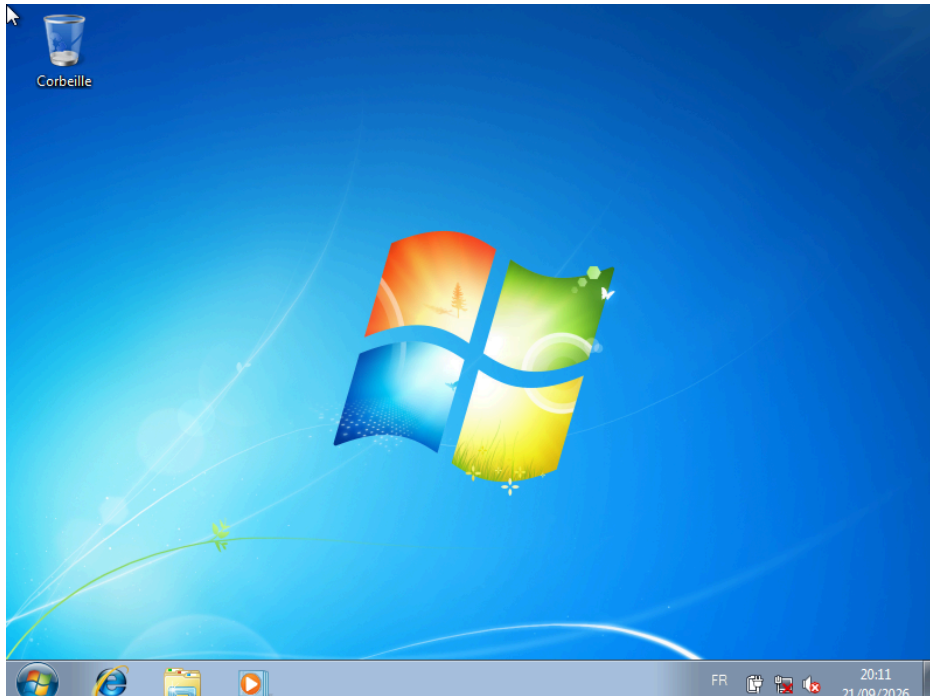
Préparation de la machine victime (Windows 7)

Kali est déjà prêt depuis le **TP1**, on prépare donc la victime. On télécharge l'**ISO** de **Windows 7 SP1**, puis on crée une nouvelle machine virtuelle et on lui associe cet **ISO**.

On choisie ce qu'on veut alloué à la machine, à modifier selon la puissance de l'ordinateur : **2 Go** de RAM, **2 processeurs** et un disque de **40 Go**. Puis on démarre la **VM** :



L'installation se déroule toute seule et la machine redémarre plusieurs fois. Une fois terminé, on arrive sur le bureau de **Windows 7** :

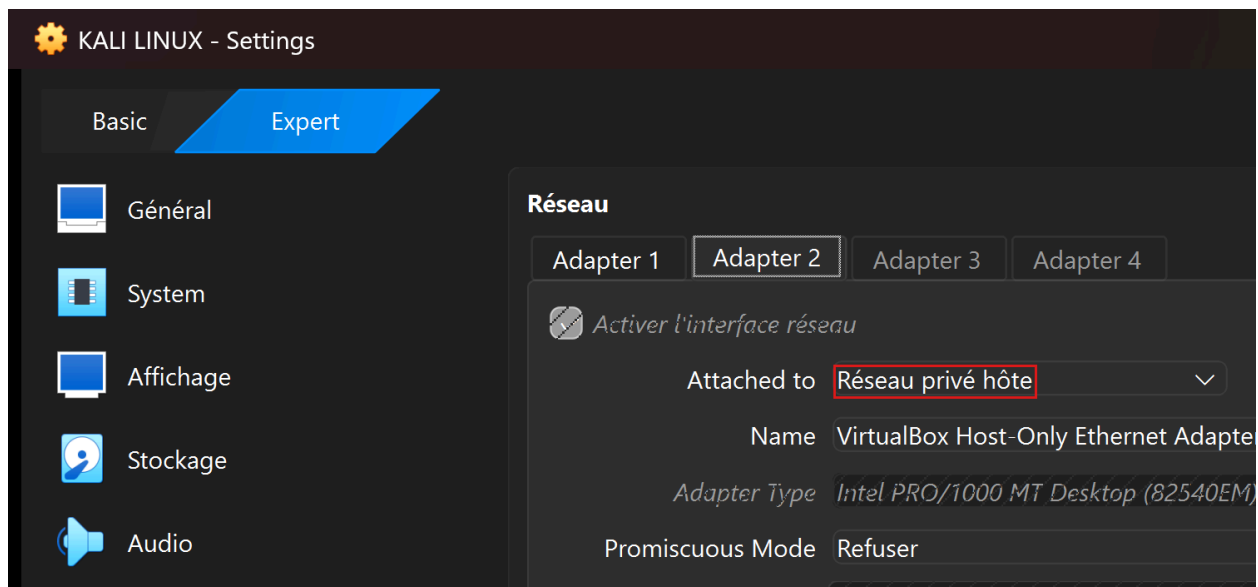


Point important pour la suite : on désactive le **pare-feu** de **Windows**, sinon la victime ne répondra pas au **ping** (il bloque l'**ICMP** par défaut).

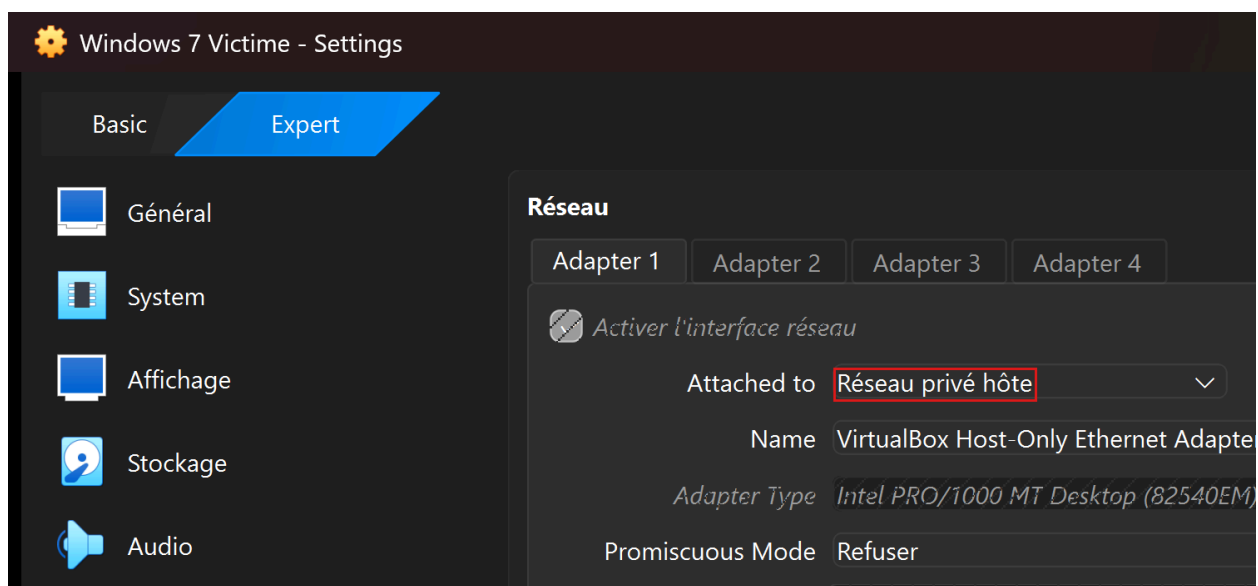
Vérification du réseau entre les deux machines

Pour que l'attaquant et la victime communiquent, il faut qu'ils soient dans le même réseau. On ajoute donc aux deux machines une carte en mode “**Réseau privé hôte**” (**Host-Only**), sur le sous-réseau **192.168.56.0/24**.

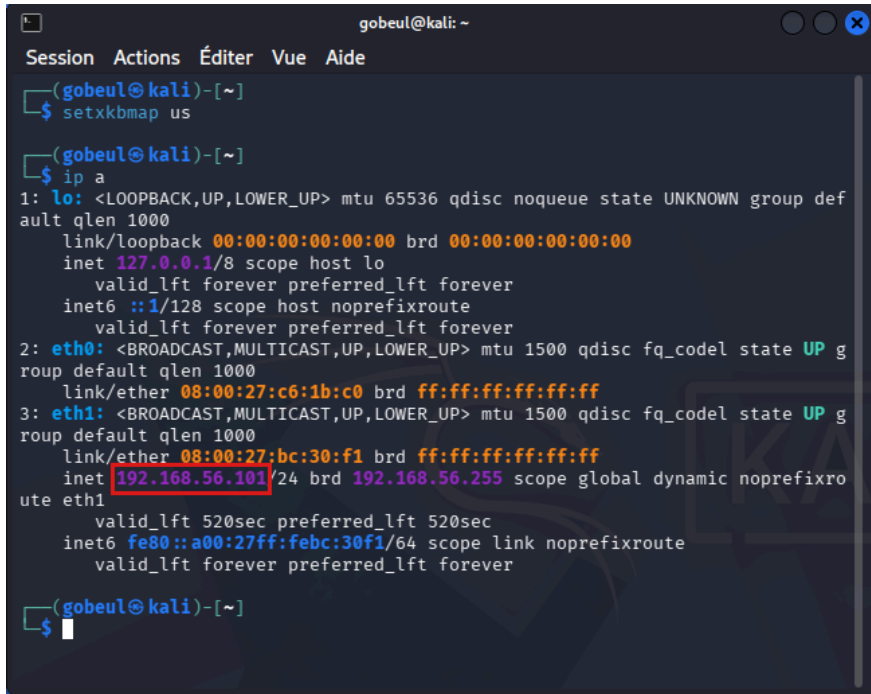
Sur **Kali**, la 2^e carte (**Adapter 2**) est bien en “**Réseau privé hôte**”. La 1^{re} reste en **NAT** pour Internet :



Sur la victime **Windows 7**, c'est configuré de la même manière :



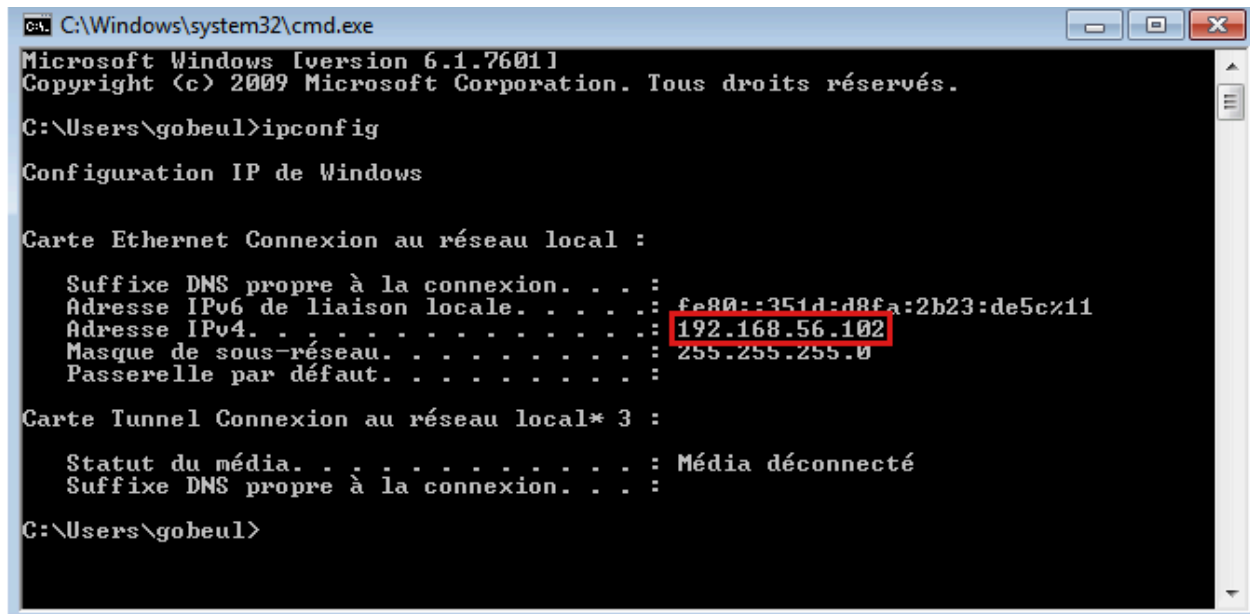
On vérifie maintenant l'adresse IP de **Kali** avec la commande **ip a** :



```
gobeul@kali: ~  
Session Actions Éditer Vue Aide  
(gobeul@kali)-[~]  
$ setxkbmap us  
(gobeul@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:c6:1b:c0 brd ff:ff:ff:ff:ff:ff  
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:bc:30:f1 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1  
        valid_lft 520sec preferred_lft 520sec  
    inet6 fe80::a00:27ff:febc:30f1/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
(gobeul@kali)-[~]  
$
```

Kali a bien récupéré l'adresse **192.168.56.101**.

On vérifie ensuite celle de la victime avec **ipconfig** :



```
C:\Windows\system32\cmd.exe  
Microsoft Windows [version 6.1.7601]  
Copyright (c) 2009 Microsoft Corporation. Tous droits réservés.  
  
C:\Users\gobeul>ipconfig  
  
Configuration IP de Windows  
  
Carte Ethernet Connexion au réseau local :  
    Suffixe DNS propre à la connexion. . . . :  
    Adresse IPv6 de liaison locale. . . . . : fe80::351d:d8fa:2b23:de5c%11  
    Adresse IPv4. . . . . : 192.168.56.102  
    Masque de sous-réseau. . . . . : 255.255.255.0  
    Passerelle par défaut. . . . . :  
  
Carte Tunnel Connexion au réseau local* 3 :  
    Statut du média. . . . . : Média déconnecté  
    Suffixe DNS propre à la connexion. . . . :  
  
C:\Users\gobeul>
```

Windows 7 a l'adresse **192.168.56.102**, avec le même masque (**255.255.255.0**).
Les deux machines sont donc bien dans le même réseau.

Il reste à vérifier qu'elles se “voient”. On fait un **ping** depuis la victime vers **Kali** :

```
C:\Windows\system32\cmd.exe
Suffixe DNS propre à la connexion. . . . :
Adresse IPv6 de liaison locale. . . . : fe80::351d:d8fa:2b23:de5c%11
Adresse IPv4. . . . . : 192.168.56.102
Masque de sous-réseau. . . . . : 255.255.255.0
Passerelle par défaut. . . . . :

Carte Tunnel Connexion au réseau local* 3 :
Statut du média. . . . . : Média déconnecté
Suffixe DNS propre à la connexion. . . . :

C:\Users\gobeul>ping 192.168.56.101

Envoi d'une requête 'Ping' 192.168.56.101 avec 32 octets de données :
Réponse de 192.168.56.101 : octets=32 temps=2 ms TTL=64
Réponse de 192.168.56.101 : octets=32 temps=1 ms TTL=64
Réponse de 192.168.56.101 : octets=32 temps<1ms TTL=64
Réponse de 192.168.56.101 : octets=32 temps=1 ms TTL=64

Statistiques Ping pour 192.168.56.101:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%)
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 2ms, Moyenne = 1ms

C:\Users\gobeul>
```

Le ping fonctionne, avec **0 % de perte**.

Puis on fait le ping dans l'autre sens, depuis **Kali** vers la victime :

```
gobeul@kali: ~
Session Actions Éditer Vue Aide
roup default qlen 1000
link/ether 08:00:27:c6:1b:c0 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
link/ether 08:00:27:bc:30:f1 brd ff:ff:ff:ff:ff:ff
inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixro
ute eth1
    valid_lft 520sec preferred_lft 520sec
    inet6 fe80::a00:27ff:febc:30f1/64 scope link noprefixroute
    valid_lft forever preferred_lft forever

(gobeul@kali)-[~]
$ n

(gobeul@kali)-[~]
$ ping -c 4 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data:
64 bytes from 192.168.56.102: icmp_seq=1 ttl=128 time=2.26 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=128 time=1.05 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=128 time=1.11 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=128 time=0.729 ms

— 192.168.56.102 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 0.729/1.287/2.260/0.579 ms

(gobeul@kali)-[~]
$
```

Ça fonctionne aussi : les deux machines communiquent bien.

Phase I : la reconnaissance

Les machines sont prêtes, on passe à la première étape : la **reconnaissance**. Le but est de **récolter le maximum d'informations** sur la cible. On la divise en deux parties.

a) La reconnaissance passive : une récolte qui **n'interagit pas avec la cible**. On cherche seulement ce qui est **public** : moteurs de recherche, réseaux sociaux, forums...

On y trouve des **emails**, des **noms d'utilisateurs**, parfois des **mots de passe** via les bases de fuites, et les **technologies utilisées** par l'entreprise. Tout ça servira à **l'exploitation**.

L'**avantage**, c'est que le potentiel est **illimité**. L'**inconvénient**, c'est que les résultats restent **limités**.

b) La reconnaissance active : là, on **interagit directement avec la cible**, avec des outils comme **nmap**. On récupère les **protocoles**, les **services** et les **ports** ouverts.

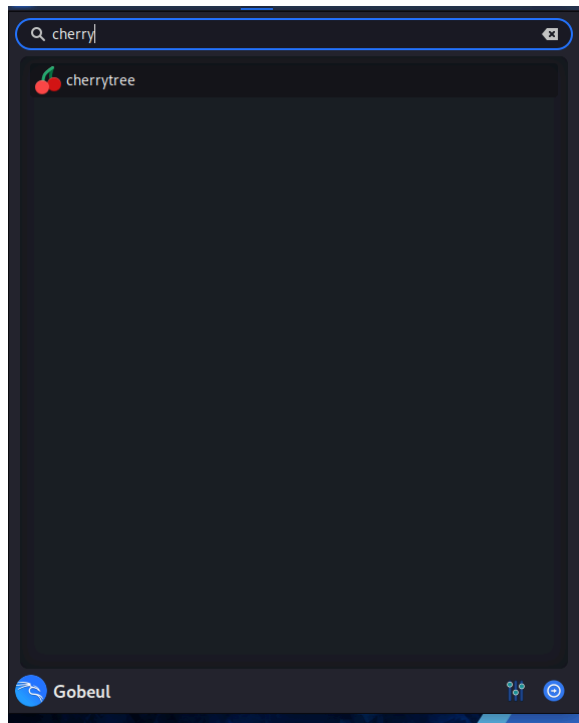
L'**avantage**, c'est qu'on obtient **beaucoup plus d'informations**. L'**inconvénient**, c'est qu'on peut se faire **bloquer** par un **pare-feu**, ou voir son **IP** bannie.

La documentation avec CherryTree

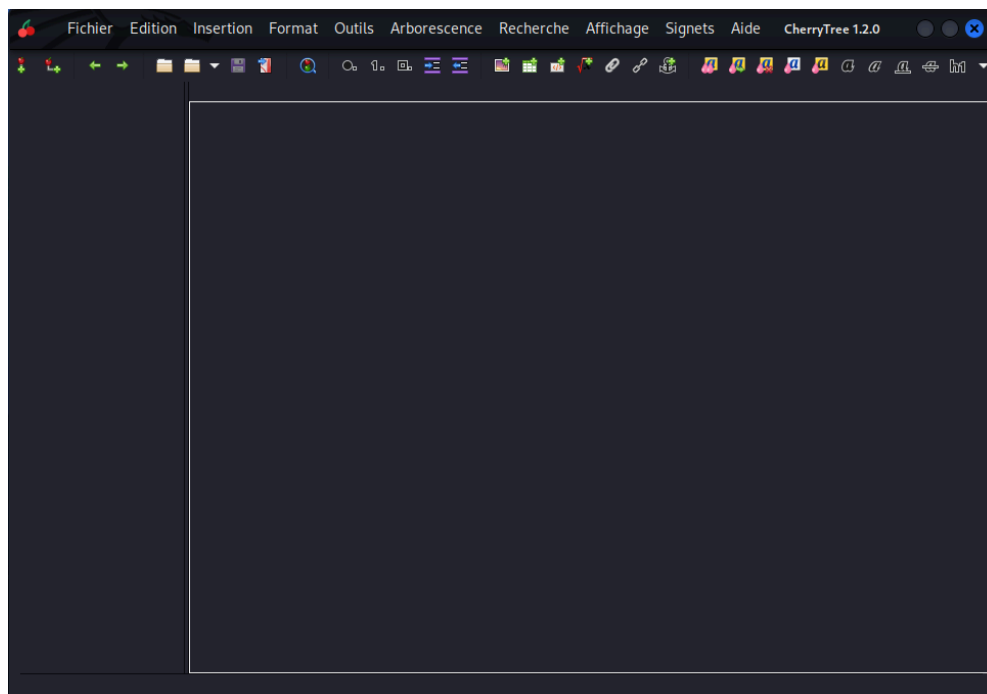
Pendant tout le test, il faut **documenter pas à pas** : les **commandes**, les **outils**, les **résultats**. Plus c'est organisé, plus le **rapport** final sera simple à écrire.

On peut utiliser un simple fichier texte, mais **Kali** intègre un outil fait pour ça : **CherryTree**.

Exercice 3 : Démarrer Kali, rechercher **CherryTree** puis le lancer.

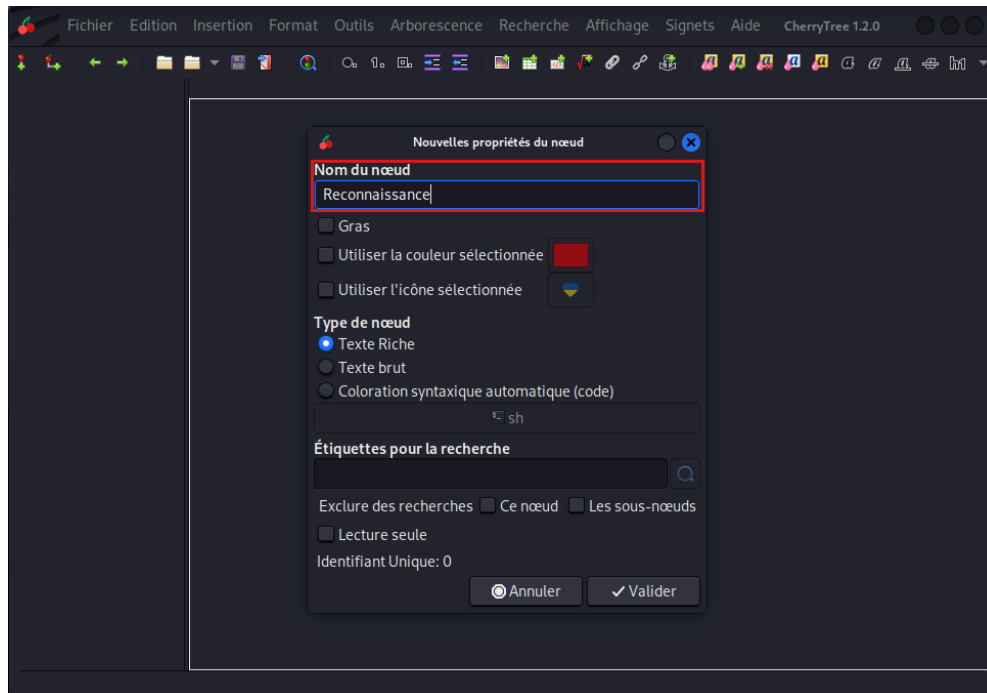


CherryTree s'ouvre sur un document vide :



Exercice 4 : Créer une section **Reconnaissance.**

On ajoute un nœud et on le nomme **Reconnaissance** :



On pourra y mettre le résultat d'un scan **nmap**, une séparation, puis celui de **nikto**. À la première modification, **CherryTree** demande d'enregistrer : on le nomme **rapport.ctb** par simplicité.

Exercice 5 : Créer la section **Exploitation.**

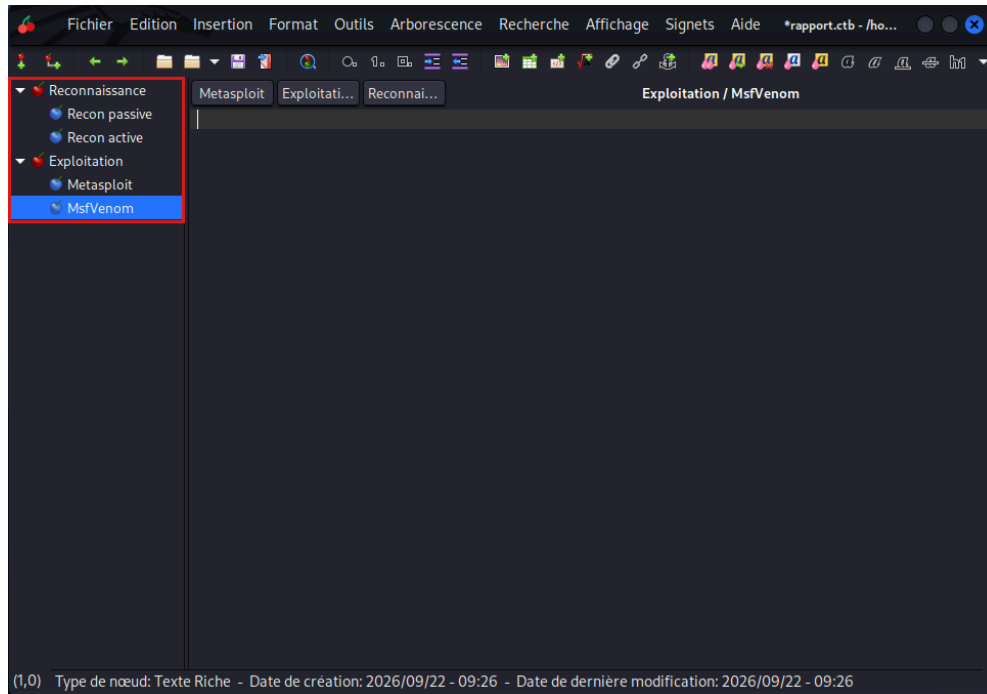
De la même façon, on ajoute un second nœud principal que l'on nomme **Exploitation**, à côté de **Reconnaissance**.

Exercice 6 : Créer l'arborescence, puis déplacer **Recon passive pour obtenir la bonne hiérarchie.**

On crée les sous-nœuds avec **Ctrl+Shift+N** (**Recon passive** et **Recon active** sous **Reconnaissance**), puis on remet **Recon passive** au bon endroit par simple glisser-déposer.

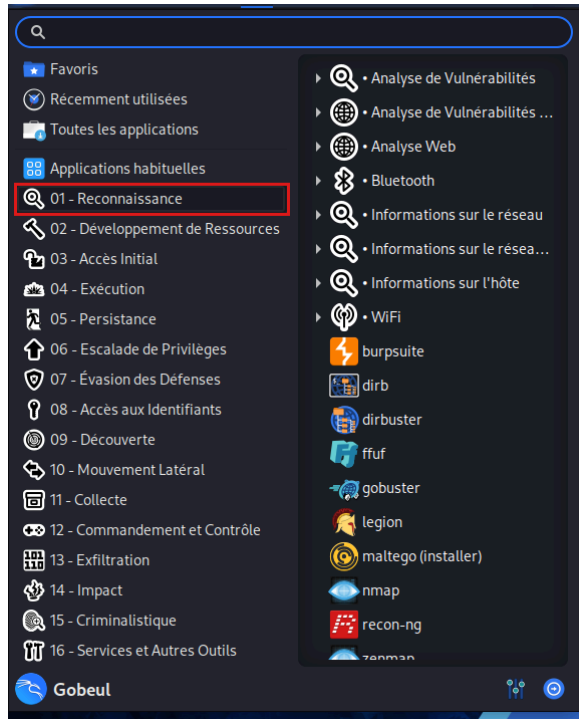
Exercice 7 : Finaliser l'arborescence.

On obtient au final l'arborescence **Reconnaissance** (**Recon passive**, **Recon active**) et **Exploitation** (**Metasploit**, **MsfVenom**) :



Les outils de reconnaissance de Kali

Sur **Kali**, les outils de reconnaissance sont dans le premier menu. Attention : le sujet parle de **01 - Récupération d'informations**, mais **Kali a réorganisé son menu** depuis. Sur notre version (**Kali 2026.2**), il s'appelle **01 - Reconnaissance** :



Les sous-catégories ont aussi été regroupé (**Informations sur le réseau, Informations sur l'hôte, Analyse Web...**), mais on y retrouve les mêmes outils, classés par protocole : **DNS, SMB, SMTP, SNMP, SSL**, plus les outils **OSINT** et de scan réseau comme **nmap**.

Plus on a d'informations sur une cible, plus on a d'**options d'exploitation** et de **recommandations** à donner au client.

Exercice 8 : Sous **Kali**, en mode console, pinguer la machine **Windows**.

Répond-elle ?

Réponse : chez nous **oui**, elle répond (**4 paquets, 0 % de perte**, voir plus haut).

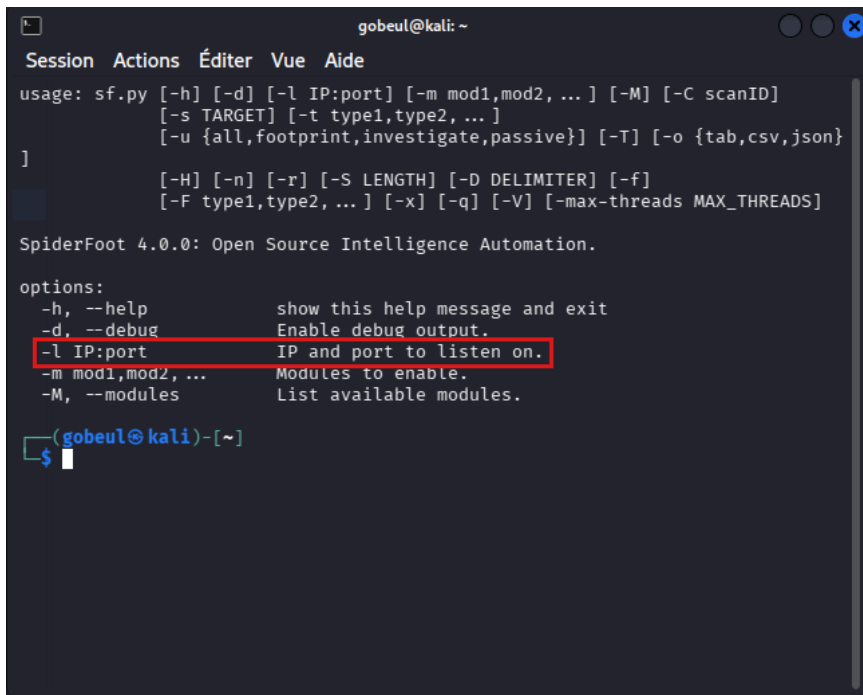
Mais c'est parce qu'on a **désactivé le pare-feu**. Par défaut, **Windows** ne répond **pas** au ping.

Microsoft bloque l'**ICMP**, car le ping sert justement aux testeurs à repérer les services et les ports d'une machine. Une machine muette n'est donc **pas forcément inactive**.

Reconnaissance avec SpiderFoot

SpiderFoot est installé par défaut sur **Kali**. Il automatise la récolte d'informations, en **passif** comme en **actif**, et stocke tout dans sa base.

On le lance d'abord sans option pour voir son aide :

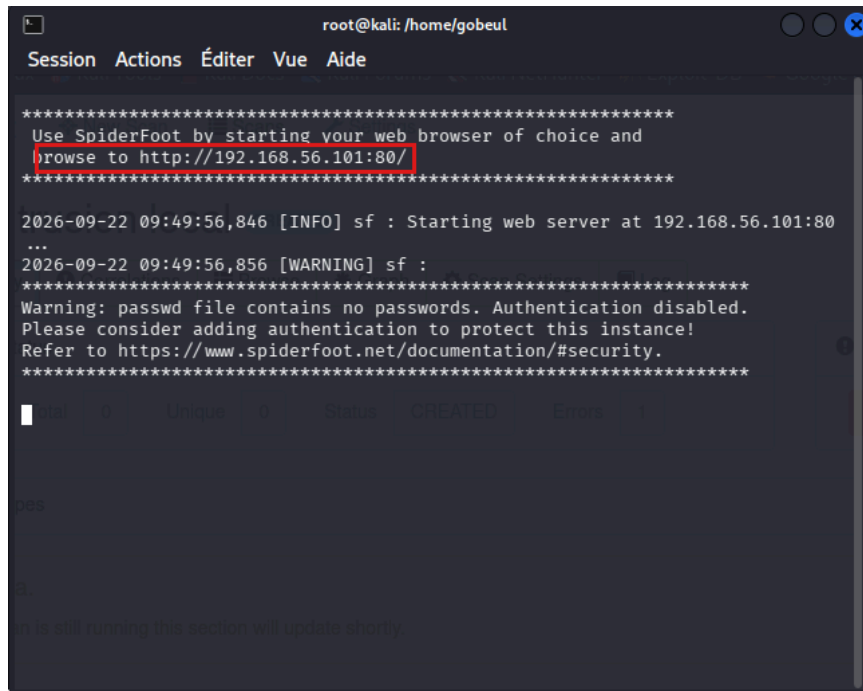


```
gobeul@kali: ~  
Session Actions Éditer Vue Aide  
usage: sf.py [-h] [-d] [-l IP:port] [-m mod1,mod2, ...] [-M] [-C scanID]  
            [-s TARGET] [-t type1,type2, ...]  
            [-u {all,footprint,investigate,passive}] [-T] [-o {tab,csv,json}]  
            [-H] [-n] [-r] [-S LENGTH] [-D DELIMITER] [-f]  
            [-F type1,type2, ...] [-x] [-q] [-V] [-max-threads MAX_THREADS]  
  
SpiderFoot 4.0.0: Open Source Intelligence Automation.  
  
options:  
-h, --help            show this help message and exit  
-d, --debug           Enable debug output.  
-l IP:port            IP and port to listen on.  
-m mod1,mod2, ...    Modules to enable.  
-M, --modules         List available modules.  
  
(gobeul@kali)-[~]  
$
```

Exercice 9 : Que signifie l'option **-l** ?

Réponse : **-l IP:port** correspond à “IP and port to listen on”, donc l'**adresse IP et le port d'écoute**. SpiderFoot s'utilise depuis une **interface web** : il faut lui dire où la publier.

On doit d'abord passer en **root** avec **sudo su**, puis on lance **spiderfoot -l 192.168.56.101:80** :



```
root@kali: /home/gobeul
Session Actions Éditer Vue Aide

*****
Use SpiderFoot by starting your web browser of choice and
browse to http://192.168.56.101:80/
*****

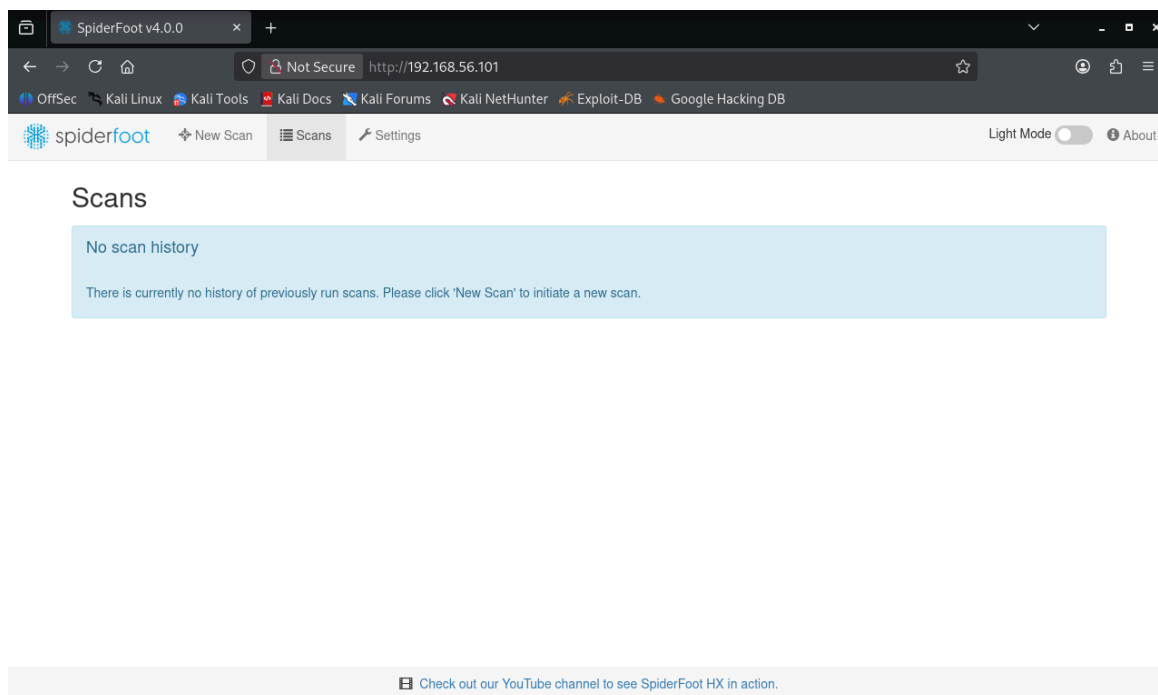
2026-09-22 09:49:56,846 [INFO] sf : Starting web server at 192.168.56.101:80
...
2026-09-22 09:49:56,856 [WARNING] sf :
*****
Warning: passwd file contains no passwords. Authentication disabled.
Please consider adding authentication to protect this instance!
Refer to https://www.spiderfoot.net/documentation/#security.
*****

Total: 0 Unique: 0 Status: CREATED Errors: 0

000

It is still running this section will update shortly.
```

Le terminal reste occupé en arrière-plan. On ouvre ensuite **http://192.168.56.101:80/** dans le navigateur :



Dans **New Scan**, on met le **Scan Name** et le **Scan Target** (un domaine, une IP ou un réseau). Trois onglets permettent de choisir ce que l'on cherche :

By Use Case propose l'option **All**, qui active tous les modules : c'est le plus complet mais le plus lent. **By Required Data** liste les informations recherchées, et **By Module** liste les modules, c'est-à-dire les **bases publiques** que SpiderFoot interroge.

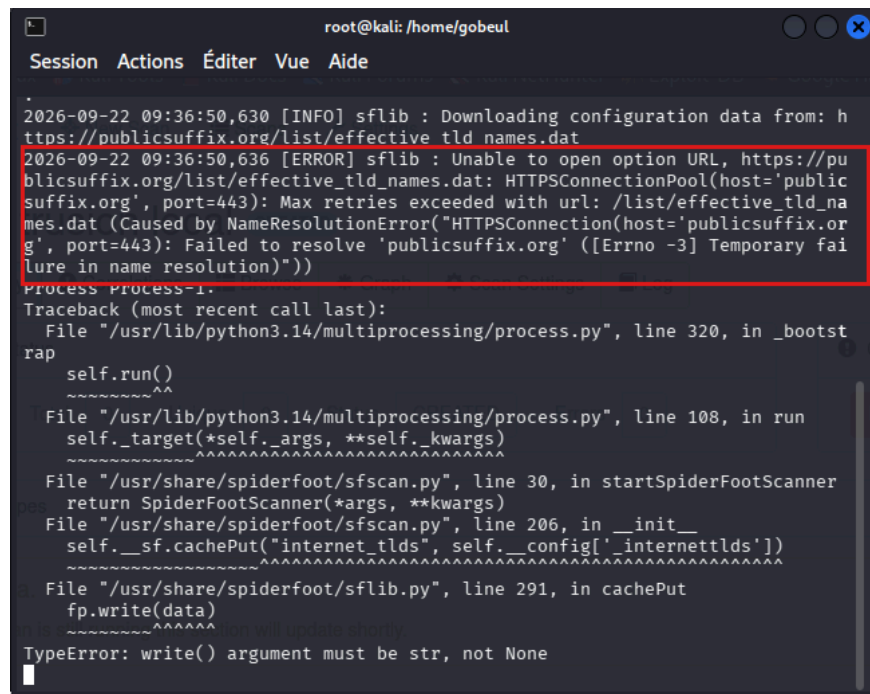
La page **New Scan** se présente ainsi :

The screenshot shows the SpiderFoot v4.0.0 web interface in a browser. The address bar shows the URL `http://192.168.56.101/newscan`. The page has a navigation bar with links for 'New Scan', 'Scans', and 'Settings'. The main content area is titled 'New Scan' and contains the following elements:

- Scan Name**: A text input field with the placeholder 'The name of this scan.'
- Scan Target**: A text input field with the placeholder 'The target of your scan.'
- Target Format Examples**: A box listing various target formats that SpiderFoot can detect:
 - Domain Name: e.g. `example.com`
 - IPv4 Address: e.g. `1.2.3.4`
 - IPv6 Address: e.g. `2606:4700:4700:1111`
 - Hostname/Sub-domain: e.g. `abc.example.com`
 - Subnet: e.g. `1.2.3.0/24`
 - Bitcoin Address: e.g. `1HesYJSP1QqcyPEjnQ9vzBL1wujruNGe7R`
 - E-mail address: e.g. `bob@example.com`
 - Phone Number: e.g. `+12345678901` (E.164 format)
 - Human Name: e.g. `"John Smith"` (must be in quotes)
 - Username: e.g. `"jsmith2000"` (must be in quotes)
 - Network ASN: e.g. `1234`
- By Use Case**: Three tabs: 'By Use Case' (selected), 'By Required Data', and 'By Module'. Under 'By Use Case', there are three radio button options:
 - All** (selected): 'Get anything and everything about the target.' Description: 'All SpiderFoot modules will be enabled (slow) but every possible piece of information about the target will be obtained and analysed.'
 - Footprint**: 'Understand what information this target exposes to the Internet.' Description: 'Gain an understanding about the target's network perimeter, associated identities and other information that is obtained through a lot of web crawling and search engine use.'
 - Investigate**: 'Best for when you suspect the target to be malicious but need more information.'
- Footer**: A link that says 'Learn about the difference between SpiderFoot and SpiderFoot HX.'

Un obstacle : l'accès à Internet et le proxy

Au premier lancement, le scan est resté bloqué sur **CREATED** avec **0 élément**. Le terminal donne la raison : SpiderFoot n'arrive pas à télécharger sa liste de domaines et plante au démarrage :

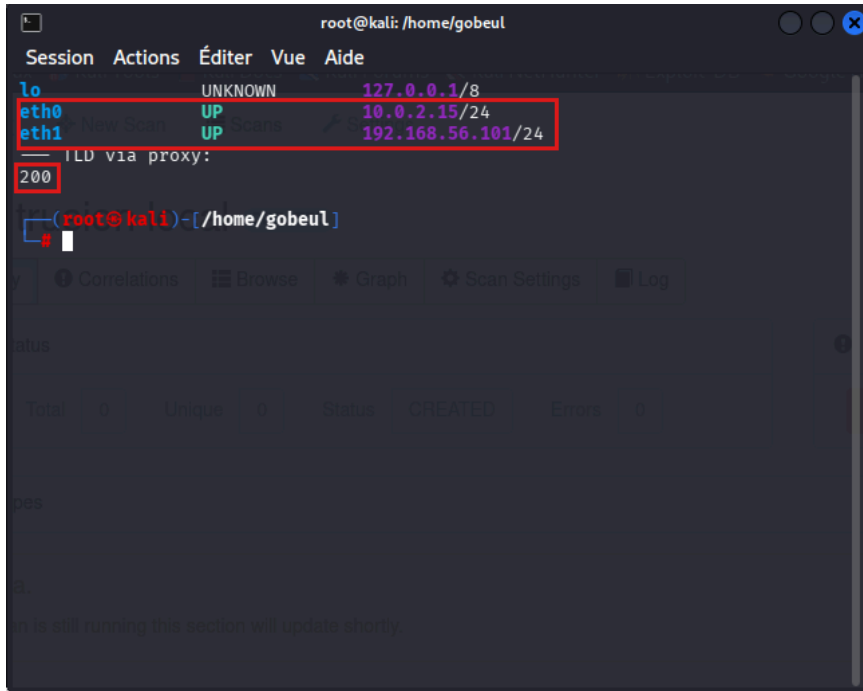


```
root@kali: /home/gobeul
Session Actions Éditer Vue Aide
2026-09-22 09:36:50,630 [INFO] sflib : Downloading configuration data from: https://publicsuffix.org/list/effective_tld_names.dat
2026-09-22 09:36:50,636 [ERROR] sflib : Unable to open option URL, https://publicsuffix.org/list/effective_tld_names.dat: HTTPSConnectionPool(host='publicsuffix.org', port=443): Max retries exceeded with url: /list/effective_tld_names.dat (Caused by NameResolutionError("HTTPSConnection(host='publicsuffix.org', port=443): Failed to resolve 'publicsuffix.org' ([Errno -3] Temporary failure in name resolution)))
Process Process-1:
Traceback (most recent call last):
  File "/usr/lib/python3.14/multiprocessing/process.py", line 320, in _bootstrap
    self.run()
    ~~~~~^^
  File "/usr/lib/python3.14/multiprocessing/process.py", line 108, in run
    self._target(*self._args, **self._kwargs)
    ~~~~~^
  File "/usr/share/spiderfoot/sfscan.py", line 30, in startSpiderFootScanner
    return SpiderFootScanner(*args, **kwargs)
  File "/usr/share/spiderfoot/sfscan.py", line 206, in __init__
    self.__sf.cachePut("internet_tlds", self.__config['_internet_tlds'])
    ~~~~~^
  File "/usr/share/spiderfoot/sflib.py", line 291, in cachePut
    fp.write(data)
    ~~~~~^
TypeError: write() argument must be str, not None
```

On regarde le réseau avec **ip -br a** et on comprend : la carte **eth0 (NAT)** n'a **aucune adresse IPv4** et il n'y a **aucune route par défaut**. La **VM** n'avait donc pas Internet, seulement le réseau de lab.

On active la carte **NAT** avec **nmcli device connect eth0**. Kali reçoit alors l'adresse **10.0.2.15** et une route par défaut.

Mais au lycée, il faut en plus passer par le **proxy** de l'établissement. Avec les deux, le test renvoie bien **200** :



The screenshot shows the SpiderFoot application interface. At the top, there's a menu bar with 'Session', 'Actions', 'Éditer', 'Vue', and 'Aide'. Below the menu, a table lists network interfaces: 'lo' (UNKNOWN, 127.0.0.1/8), 'eth0' (UP, 10.0.2.15/24), and 'eth1' (UP, 192.168.56.101/24). The 'eth1' row is highlighted with a red box. Below the table, a section titled 'TLD via proxy:' shows the result '200', which is also highlighted with a red box. At the bottom, there's a terminal window showing the command prompt '(root@kali) - [/home/gobeul]' and a cursor. Below the terminal, there are buttons for 'Correlations', 'Browse', 'Graph', 'Scan Settings', and 'Log'. At the very bottom, there's a status bar with 'Total: 0', 'Unique: 0', 'Status: CREATED', and 'Errors: 0'.

Il suffit alors d'exporter le proxy avant de relancer l'outil, avec **export http_proxy=http://172.16.63.130:3128**. Cette fois, SpiderFoot démarre sans erreur.

Exercice 10 : Trouver le site **HackerOne**, puis rédiger en quelques lignes l'activité de ce site.

HackerOne (hackerone.com) est une plateforme de **bug bounty**. Elle met en relation des **entreprises** et des **hackers éthiques**.

Chaque entreprise publie un **programme** avec le périmètre autorisé. Les chercheurs y signalent les failles qu'ils trouvent et sont **récompensés** selon la criticité.

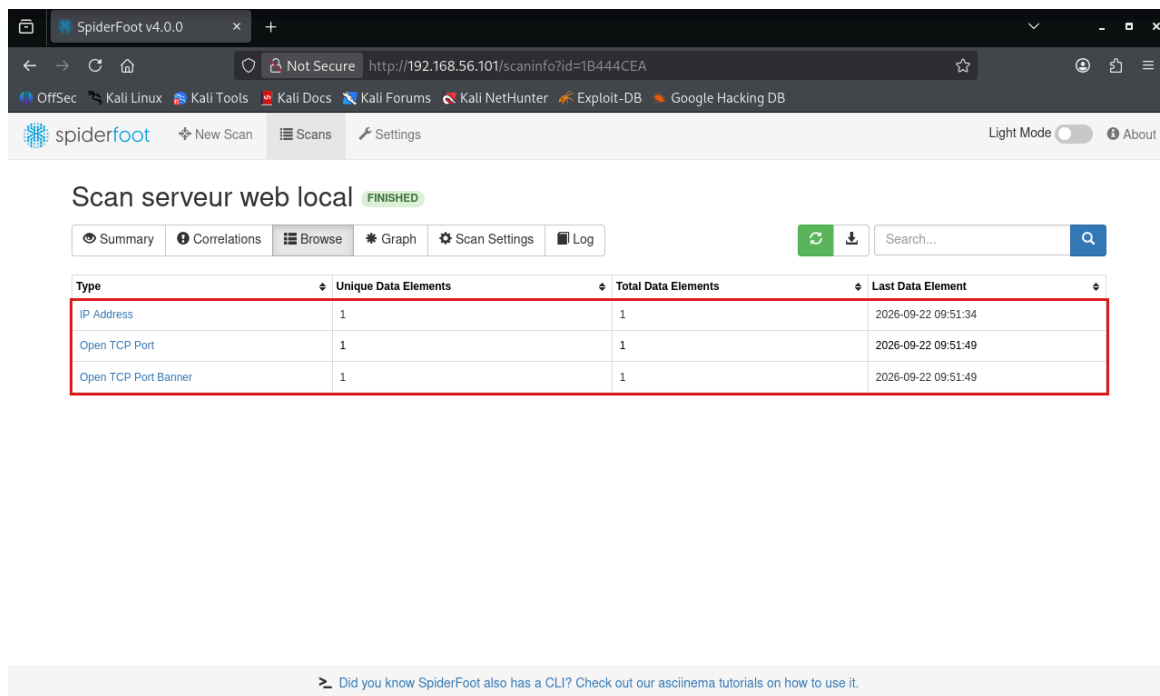
SpiderFoot a d'ailleurs un module qui interroge HackerOne sur la cible.

Exercice 11 : Effectuer un scan sur un **serveur web local** et noter les informations récoltées.

Attention : un scan ne doit **jamais** être lancé sur une cible qui ne nous appartient pas, il peut la faire tomber.

On ne travaille donc que sur son propre matériel. Ici la cible est notre **Kali (192.168.56.101)**, qui héberge le serveur web.

Le scan passe cette fois en **RUNNING**, puis **FINISHED**. Les informations récoltées sont dans l'onglet **Browse** :



Type	Unique Data Elements	Total Data Elements	Last Data Element
IP Address	1	1	2026-09-22 09:51:34
Open TCP Port	1	1	2026-09-22 09:51:49
Open TCP Port Banner	1	1	2026-09-22 09:51:49

Le scan a trouvé trois types d'informations. **IP Address** donne l'adresse IP de la cible. **Open TCP Port** indique un **port ouvert**, ici le port **80** du serveur web. **Open TCP Port Banner** donne la **bannière** de ce port, qui identifie le service.

Le résultat est volontairement limité : la cible est une **IP privée**, donc les modules **OSINT** n'ont rien à trouver dessus.

Sur un vrai nom de domaine, le même scan remonterait des centaines d'éléments.

Phase II : le scanning

Après la reconnaissance passive avec **SpiderFoot**, on passe à la **reconnaissance active**, aussi appelée le **scanning**. Cette fois on **interagit directement avec la cible** pour récupérer ses **ports ouverts**, ses **services** et son **système d'exploitation**.

On utilise deux outils : **Nmap**, qui cartographie les ports et les services, et **Nessus**, qui cherche les **vulnérabilités**.

Exercice 12 : Rechercher ce qu'est **httpd**.

httpd est le **démon** (le service qui tourne en arrière-plan) du serveur web **Apache**. C'est lui qui écoute les requêtes **HTTP** et renvoie les pages. Le “**d**” final signifie ***daemon***.

Le scanner de ports Nmap

Nmap est préinstallé sur **Kali** ; c'est l'outil de scan le plus utilisé pour les tests d'intrusion. On commence par élever les privilèges avec **sudo su**, car certains scans en ont besoin.

Exercice 13 : Scanner la machine **Windows** à partir de son adresse **IP**.

On lance **nmap 192.168.56.102** :

```
(root@kali)-[/home/gobeul]
# nmap 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 17:01 +0200
Nmap scan report for 192.168.56.102
Host is up (0.0025s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
49152/tcp  open  unknown
49153/tcp  open  unknown
49154/tcp  open  unknown
49155/tcp  open  unknown
49156/tcp  open  unknown
49157/tcp  open  unknown
MAC Address: 08:00:27:9C:BC:72 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.13 seconds

(root@kali)-[/home/gobeul]
#
```

On obtient la liste des **ports ouverts** avec leurs services : **135** (msrpc), **139** (netbios-ssn) et **445** (microsoft-ds), plus une série de ports dynamiques. Ce sont les ports classiques d'une machine **Windows**. Certains services restent affichés en **unknown**.

Exercice 14 : Tester **nmap** sur le réseau local en entier.

On lance **nmap 192.168.56.0/24** pour balayer tout le sous-réseau :

```
(root@kali)-[/home/gobeul]
# nmap 192.168.56.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 17:01 +0200
Nmap scan report for 192.168.56.1
Host is up (0.00068s latency).
All 1000 scanned ports on 192.168.56.1 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 0A:00:27:00:00:0C (Unknown)

Nmap scan report for 192.168.56.100
Host is up (0.00040s latency).
All 1000 scanned ports on 192.168.56.100 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 08:00:27:E0:4F:5A (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.56.102
Host is up (0.0011s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
49152/tcp  open  unknown
49153/tcp  open  unknown
49154/tcp  open  unknown
49155/tcp  open  unknown
49156/tcp  open  unknown
49157/tcp  open  unknown
MAC Address: 08:00:27:9C:BC:72 (Oracle VirtualBox virtual NIC)

Nmap scan report for 192.168.56.101
Host is up.
All 1000 scanned ports on 192.168.56.101 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 256 IP addresses (4 hosts up) scanned in 220.40 seconds
(root@kali)-[/home/gobeul]
```

Nmap trouve **4 machines** : la passerelle **192.168.56.1**, le serveur **DHCP** de VirtualBox (**.100**), notre attaquant **Kali (.101)** et la victime **Windows 7 (.102)**.

Exercice 15 : Afficher le **système d'exploitation** de la cible avec l'argument **-O**.

On lance **nmap -O 192.168.56.102** :

```
(root@kali)-[/home/gobeul]
# nmap -O 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 17:09 +0200
Nmap scan report for 192.168.56.102
Host is up (0.0016s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
49152/tcp  open  unknown
49153/tcp  open  unknown
49154/tcp  open  unknown
49155/tcp  open  unknown
49156/tcp  open  unknown
49157/tcp  open  unknown
MAC Address: 08:00:27:9C:BC:72 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Microsoft Windows 2008|7|Vista|8.1
OS CPE: cpe:/o:microsoft:windows_server_2008:r2 cpe:/o:microsoft:windows_7 cpe:/o:microsoft:windows_vista cpe:/o:microsoft:windows_8.1
OS details: Microsoft Windows Vista SP2 or Windows 7 or Windows Server 2008 R2 or Windows 8.1
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.92 seconds
(root@kali)-[/home/gobeul]
```

Nmap ne donne pas une réponse exacte mais une **fourchette** : **Windows Vista, 7, Server 2008 R2 ou 8.1**. C'est déjà une bonne indication.

Exercice 16 : Afficher les **versions des services** avec l'argument **-sV**.

On lance **nmap -sV 192.168.56.102**. Cela prend un peu plus de temps :

```
(root@kali)-[/home/gobeul]
# nmap -sV 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 17:10 +0200
Nmap scan report for 192.168.56.102
Host is up (0.0015s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
49152/tcp  open  msrpc        Microsoft Windows RPC
49153/tcp  open  msrpc        Microsoft Windows RPC
49154/tcp  open  msrpc        Microsoft Windows RPC
49155/tcp  open  msrpc        Microsoft Windows RPC
49156/tcp  open  msrpc        Microsoft Windows RPC
49157/tcp  open  msrpc        Microsoft Windows RPC
MAC Address: 08:00:27:9C:BC:72 (Oracle VirtualBox virtual NIC)
Service Info: Host: WIN7-VICTIME; OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 59.93 seconds

(root@kali)-[/home/gobeul]
```

Cette fois on a les **versions** : **Microsoft Windows RPC** sur les ports RPC, et surtout le port **445** identifié comme **Microsoft Windows 7 - 10**. On récupère aussi le **nom de la machine** : **WIN7-VICTIME**.

Exercice 17 : Lancer un **scan agressif (-A)** sur la victime.

Le scan **-A** combine la détection d'**OS**, des **versions**, des **scripts** et un **traceroute**. Il donne beaucoup d'informations, mais il est **bruyant** : à ne pas utiliser sur un réseau de production. On lance **nmap -A 192.168.56.102** :

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-22 17:11 +0200
Nmap scan report for 192.168.56.102
Host is up (0.0016s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
135/tcp    open  msrpc          Microsoft Windows RPC
139/tcp    open  netbios-ssn    Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds   Windows 7 Ultimate 7601 Service Pack 1 microsoft-ds (workgroup: WORKGROUP)
49152/tcp  open  msrpc          Microsoft Windows RPC
49153/tcp  open  msrpc          Microsoft Windows RPC
49154/tcp  open  msrpc          Microsoft Windows RPC
49155/tcp  open  msrpc          Microsoft Windows RPC
49156/tcp  open  msrpc          Microsoft Windows RPC
49157/tcp  open  msrpc          Microsoft Windows RPC
MAC Address: 08:00:27:9C:BC:72 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Microsoft Windows 2008|7|Vista|8.1
OS CPE: cpe:/o:microsoft:windows_server_2008:r2 cpe:/o:microsoft:windows_7 cpe:/o:microsoft:windows_vista cpe:/o:microsoft:windows_8.1
OS details: Microsoft Windows Vista SP2 or Windows 7 or Windows Server 2008 R2 or Windows 8.1
Network Distance: 1 hop
Service Info: Host: WIN7-VICTIME; OS: Windows; CPE: cpe:/o:microsoft:windows

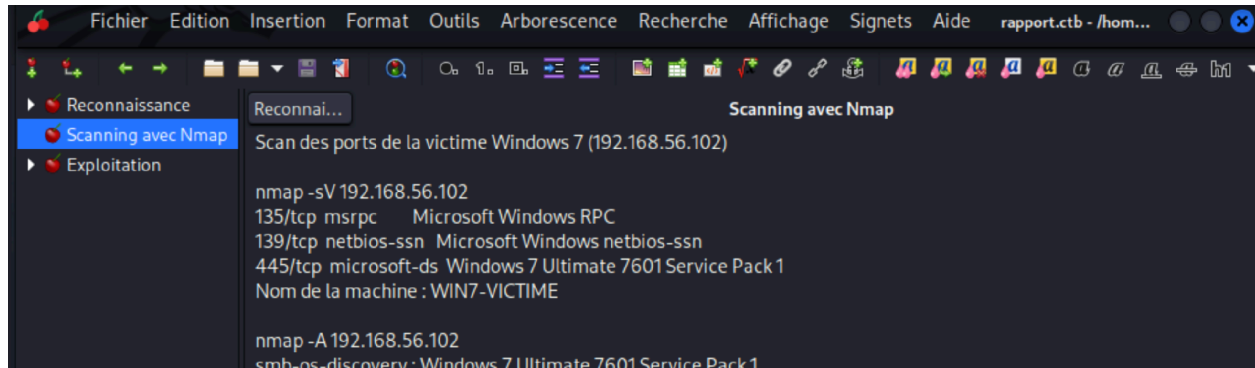
Host script results:
|_ smb-os-discovery:
|_   OS: Windows 7 Ultimate 7601 Service Pack 1 (Windows 7 Ultimate 6.1)
|_   OS CPE: cpe:/o:microsoft:windows_7::sp1
|_   Computer name: WIN7-VICTIME
|_   NetBIOS computer name: WIN7-VICTIME\x00
|_   Workgroup: WORKGROUP\x00
|_   System time: 2026-09-21T23:13:18+02:00
```

Grâce au script **smb-os-discovery**, la version est maintenant **précise** : **Windows 7 Ultimate 7601 Service Pack 1**. Le numéro de **Service Pack** est une information importante pour la suite. On voit aussi que la signature des messages **SMB** est désactivée.

Nmap a encore d'autres options utiles : **--script** lance des **scripts** (le moteur NSE) pour des tâches avancées, comme tester des services vulnérables ; et **-p** cible des **ports précis** au lieu de tous les scanner (par exemple **nmap -p 80,445 192.168.56.102** pour ne regarder que le web et le partage **SMB**).

Exercice 18 : Documenter le scan dans **CherryTree**.

Comme pour la reconnaissance, on garde une trace des scans. On crée dans **CherryTree** la section **Scanning avec Nmap** et on y colle le résultat des commandes :

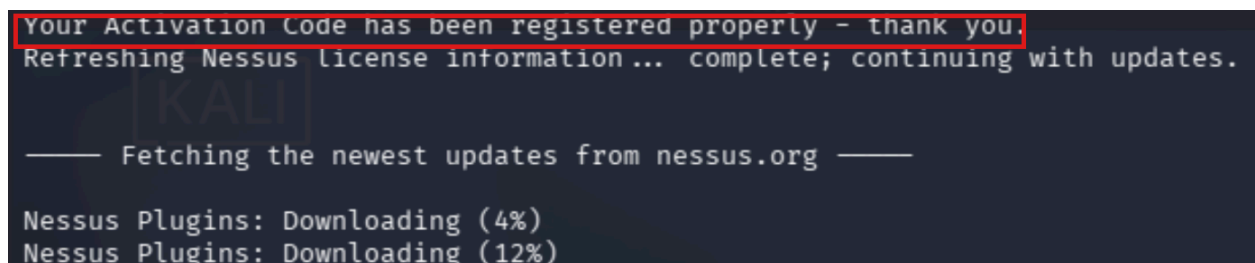


Le scanner de vulnérabilités Nessus

Nmap dit quels ports sont ouverts ; **Nessus** va plus loin : c'est un **scanner de vulnérabilités**. On lui donne une adresse, il teste des milliers de failles connues et rend un rapport classé par **criticité**.

Exercice 19 : Installer et enregistrer **Nessus** sur **Kali**.

On télécharge le paquet **.deb** sur le site de **Tenable**, on l'installe avec **dpkg -i**, puis on démarre le service avec **systemctl start nessusd**. **Nessus Essentials** est **gratuit** mais demande une **clé d'activation** reçue par mail ; on l'enregistre alors avec **nessuscli fetch --register** :



Nessus télécharge ses **plugins** (la liste des vulnérabilités à tester). Une fois prêt, le service est **actif** :

```
systemctl start nessusd 66 systemctl status nessusd --no-pager | head -8
● nessusd.service - The Nessus Vulnerability Scanner
   Loaded: loaded (/usr/lib/systemd/system/nessusd.service; disabled; preset: disabled)
   Active: active (running) since Tue 2026-09-22 17:30:14 CEST; 42ms ago
 Invocation: d2f23c7368294539955ef898b003fa8a
    Main PID: 20702 (nessus-service)
      Tasks: 1 (limit: 4617)
     Memory: 1.9M (peak: 1.9M)
        CPU: 12ms
```

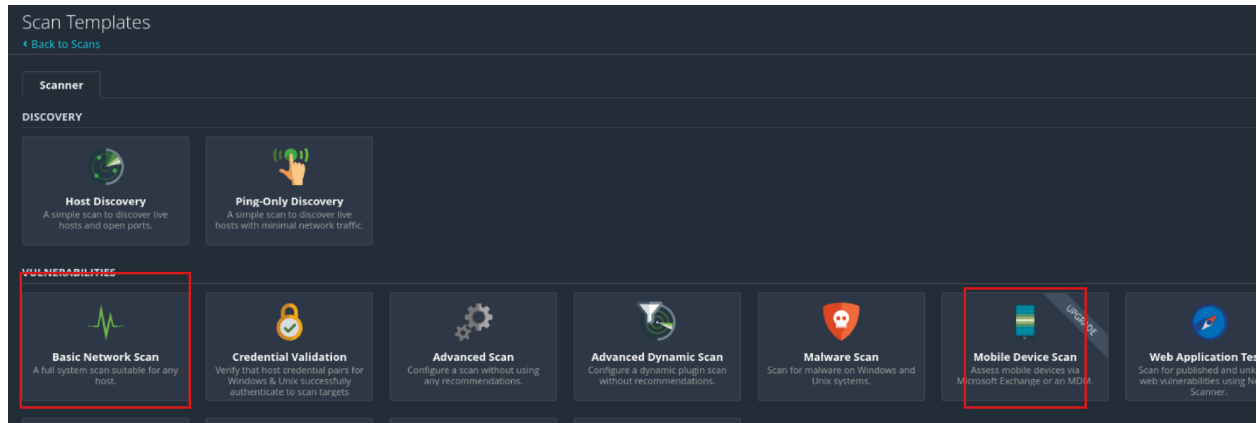
On ouvre l'interface web **https://localhost:8834** et on se connecte avec le compte administrateur :



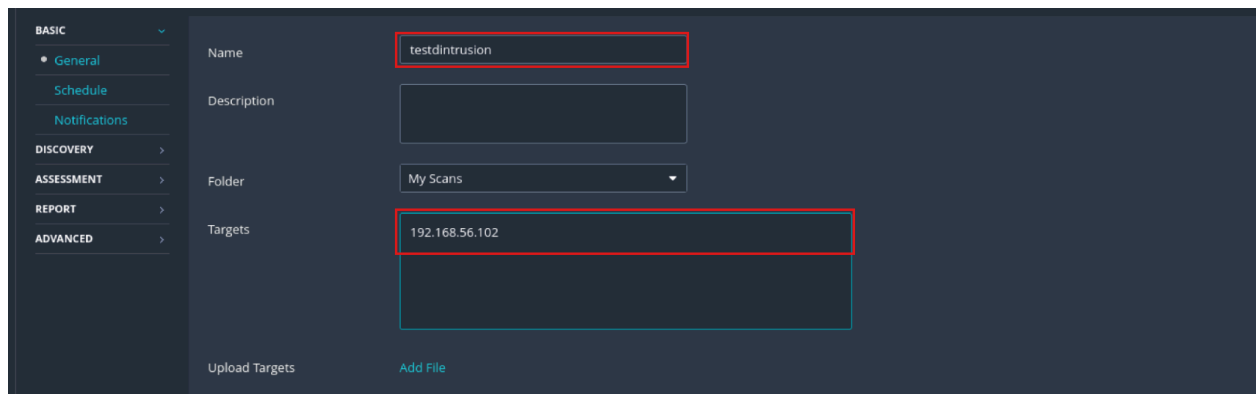
Exercice 20 : Lancer un scan sur la victime **Windows 7**.

On clique sur **New Scan** : **Nessus** propose plusieurs modèles de scan.

Réponse : le scan pour appareils mobiles s'appelle **Mobile Device Scan**. Il porte le badge **UPGRADE** : il n'est **pas disponible** dans la version gratuite (**Essentials**), il faut un serveur **MDM** et la version **Professional**.

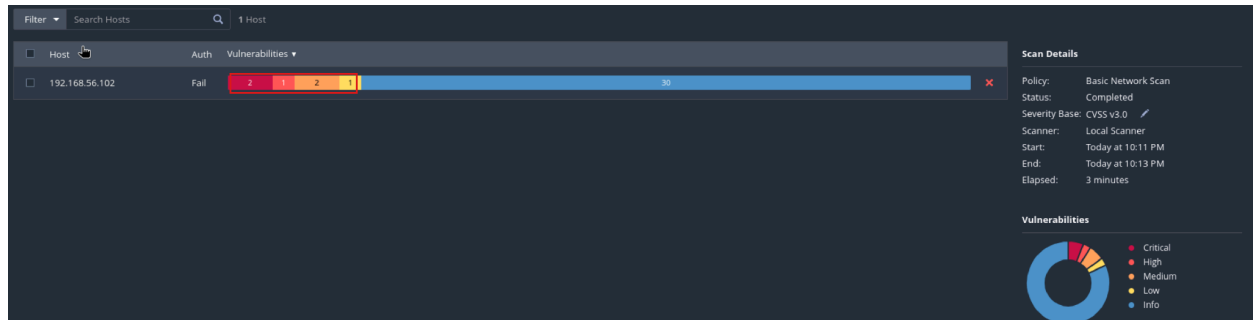


Pour la victime on choisit **Basic Network Scan**, on met un nom et la cible **192.168.56.102**, puis on lance :



Exercice 21 : Consulter et exporter le rapport de vulnérabilités.

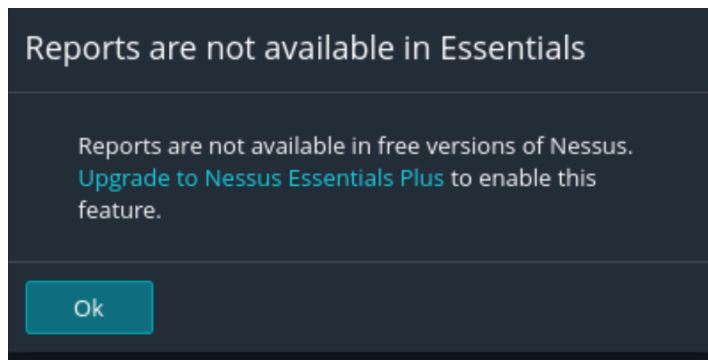
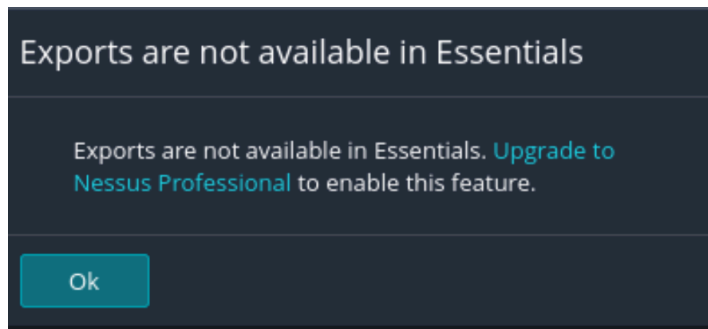
Au bout de **3 minutes**, le scan est terminé. Les vulnérabilités sont classées par **sévérité** avec un score **CVSS** : la victime présente **2 vulnérabilités critiques**, **1 haute**, **2 moyennes** et **1 basse** (plus 30 informatives) :



En détail, on retrouve surtout des problèmes **Windows** et **SMB** (le partage de fichiers, port **445**) — le point d'entrée le plus intéressant pour la suite :

Filter	Search Vulnerabilities	18 Vulnerabilities						
☐ Sev	CVSS	VPR	EPSS	Name	Family	Count		
☐ MIXED	...	...	...	 Microsoft Windows (Multiple Issues)	Windows	4	 	
☐ MIXED	...	...	...	 SMB (Multiple Issues)	Misc.	2	 	
☐ LOW	2.1 *			ICMP Timestamp Request Remote Date Disclosure	General	1	 	
☐ INFO	...	...	...	 SMB (Multiple Issues)	Windows	6	 	
☐ INFO				DCE Services Enumeration	Windows	8	 	
☐ INFO				Nessus SYN scanner	Port scanners	3	 	
☐ INFO				Common Platform Enumeration (CPE)	General	1	 	
☐ INFO				Device Type	General	1	 	
☐ INFO				Ethernet Card Manufacturer Detection	Misc.	1	 	
☐ INFO				Ethernet MAC Addresses	General	1	 	
☐ INFO				Link-Local Multicast Name Resolution (LLMNR) Detection	Service detection	1	 	

On essaie de générer le rapport PDF. Mais **ni le bouton Export ni le bouton Report** ne fonctionnent : les deux affichent que la fonction n'est **pas disponible dans Nessus Essentials** (la version gratuite) et qu'il faut passer à une version **payante** (Nessus Essentials Plus ou Professional) :



En fait, ce blocage vient de la **version de Nessus** : on a installé la **dernière version (10.12.4)**, et depuis la branche 10.12, Tenable a **retiré l'export et les rapports de la version gratuite** Essentials. C'est écrit dans leur documentation officielle : " You cannot create scan exports or reports when using Tenable Nessus Essentials ". Les **versions plus anciennes** de Nessus, elles, permettaient encore d'exporter en gratuit : c'est pour ça que l'export fonctionne sur une installation plus ancienne (comme celle utilisée pour préparer le sujet), mais plus sur une installation faite aujourd'hui.

Comme l'export est bloqué chez nous, on **documente les résultats directement depuis l'interface** (captures ci-dessus), et on s'appuie sur le **rapport PDF d'exemple** complet fourni avec le sujet (**test dintrusion.pdf**) pour la mise en forme d'un rapport type.

Conclusion

Ce **TP** nous a permis de découvrir la méthodologie d'un **test d'intrusion**, et de monter l'environnement de travail : une machine attaquante (**Kali Linux**) et une victime (**Windows 7 SP1**) dans le même réseau.

On a ensuite déroulé les deux premières phases : la **reconnaissance** (le **passif** avec **SpiderFoot**, l'**actif**, la documentation avec **CherryTree**) puis le **scanning**. Avec **Nmap** on a récupéré les **ports**, les **services** et la version exacte de la victime (**Windows 7 SP1**) ; avec **Nessus** on a listé ses **vulnérabilités**, dont **2 critiques**.

La prochaine étape sera le **gain d'accès**, en exploitant ces vulnérabilités.